

Relazione Laboratori: Complementi di Reti (2025-2026)

Documento di preparazione per l'esame orale. La relazione illustra gli obiettivi, il funzionamento generale e le risposte ai quesiti posti in ogni laboratorio.

LAB-K1: Docker hands-on

Obiettivo

L'obiettivo principale di questo primo laboratorio è familiarizzare con l'ambiente Docker e i suoi comandi base. Poiché i laboratori successivi (basati su Kathará) utilizzano Docker come motore sottostante per l'emulazione delle reti, è fondamentale comprenderne il funzionamento, il concetto di "container" e la gestione delle immagini.

Funzionamento

Il laboratorio prevede un approccio pratico (hands-on) in cui lo studente:

1. Verifica la corretta installazione di Docker eseguendo un container di test (docker run hello-world).
2. Segue un tutorial online per approfondire teoricamente la differenza tra macchine virtuali (VM) e container.
3. Impara a gestire il ciclo di vita dei container e delle immagini scaricando template (pull), avviando processi isolati (run), monitorandoli (ps) e facendo pulizia nel sistema (rm, rmi, prune).

Risposte alle Domande (Q1 - Q11)

- **Q1: Cos'è un container e in cosa differisce da una Virtual Machine (VM)? In che termini i container sono "migliori" delle VM?**

Un container è un'unità standardizzata di software che pacchettizza il codice e tutte le sue dipendenze, permettendo all'applicazione di girare in modo rapido e affidabile in ambienti diversi.

A differenza di una VM, che virtualizza l'intero hardware e richiede un sistema operativo "guest" completo gestito da un hypervisor, i container virtualizzano solo il livello del sistema operativo. Condividono il kernel del sistema operativo "host", risultando molto più leggeri, veloci da avviare e richiedendo molte meno risorse (CPU/RAM e spazio su disco) rispetto alle VM.

- **Q2: Qual è il significato di *isolamento* per le VM e per i container?**

Nelle VM, l'isolamento è garantito a livello hardware dall'hypervisor: ogni VM è una macchina a sé stante, completamente separata dalle altre. Nei container, l'isolamento è garantito a livello di processo tramite funzionalità del kernel Linux (come *namespaces* per isolare file system, reti, processi e *cgroups* per limitare le risorse hardware). È un

isolamento più "morbido" rispetto all'hardware, ma sufficiente per far credere all'applicazione di essere l'unica in esecuzione sulla macchina.

- **Q3: Qual è il significato del comando docker pull?**

Il comando serve a scaricare un'immagine (un template) da un registry (come Docker Hub) al disco locale della propria macchina, senza però avviarla.

- **Q4: Qual è il significato del comando docker images e come è correlato a docker pull?**

docker images elenca tutte le immagini Docker attualmente salvate sul disco locale. È strettamente correlato a docker pull perché mostra l'elenco di ciò che è stato precedentemente scaricato tramite quel comando.

- **Q5: Qual è il significato del comando docker run e come è correlato a docker images?**

docker run crea un nuovo container a partire da un'immagine e lo avvia. Se l'immagine richiesta non è presente nell'elenco locale (visibile con docker images), Docker eseguirà automaticamente un docker pull per scaricarla prima di avviare il container.

- **Q6: Qual è la differenza tra un'immagine e un container?**

L'immagine è un template statico, di sola lettura, che contiene il codice e le dipendenze (è paragonabile a una classe nella programmazione ad oggetti o al file eseguibile su disco). Il container è l'istanza in esecuzione di quell'immagine (paragonabile all'oggetto istanziato o al processo in esecuzione nella RAM).

- **Q7: Qual è il significato del comando docker ps e come è correlato a docker run?**

docker ps elenca i container che sono attualmente in esecuzione. È correlato a docker run perché mostra i processi istanziati e mantenuti attivi proprio da quel comando.

- **Q8: Qual è la differenza tra docker ps e docker ps -a?**

docker ps mostra solo i container attualmente in esecuzione (running). Aggiungendo il flag -a (all), vengono mostrati *tutti* i container presenti sul sistema, inclusi quelli che sono stati fermati, che hanno terminato la loro esecuzione o che sono andati in errore.

- **Q9: Qual è il significato del comando docker rm e come è correlato a docker ps?**

docker rm rimuove definitivamente uno o più container dal sistema locale. Solitamente si utilizza per rimuovere container non più attivi (visibili con docker ps -a). Per rimuovere un container in esecuzione (visibile con docker ps) occorre prima fermarlo o usare il flag -f (force).

- **Q10: Qual è il significato di docker container prune?**

È un comando di utilità che permette di fare pulizia rapida: rimuove in un solo colpo tutti i container che si trovano nello stato "stopped" (fermati), liberando spazio su disco.

- **Q11: Qual è il significato di docker rmi e come è correlato a docker container prune?**

docker rmi rimuove le *immagini* dal disco locale (mentre prune o rm rimuovono i *container*). Non è possibile fare rmi di un'immagine se c'è un container (anche fermato) che fa riferimento ad essa. Pertanto, spesso è necessario usare docker container prune prima per eliminare i container inattivi, e solo successivamente usare docker rmi per rimuovere le immagini sottostanti.

LAB-K2: Basics on Kathará

Obiettivo

Questo laboratorio introduce Kathará, un framework basato su container Docker progettato specificamente per emulare reti di computer. L'obiettivo è capire come avviare e fermare un'infrastruttura di rete emulata, comprendere la struttura di un progetto Kathará (file lab.conf e script .startup) e ripassare i concetti base di indirizzamento e routing IPv4 (come assegnare IP e ispezionare le tabelle di routing).

Funzionamento

Kathará permette di definire una topologia di rete creando dei container che simulano dei dispositivi di rete (host, router, switch). Questi dispositivi vengono collegati tra loro tramite "domini di collisione virtuali" (reti isolate).

Nel laboratorio "K2-TwoHosts", viene creata una topologia minimale:

- Due host: pc1 e pc2.
- Entrambi sono collegati allo stesso dominio di collisione chiamato "A" attraverso la loro interfaccia eth0.
- I file pc1.startup e pc2.startup contengono i comandi Linux standard (ip address add...) per assegnare gli indirizzi IPv4 (rispettivamente 10.0.0.1/24 e 10.0.0.2/24) al momento dell'avvio del container.

Inoltre, viene esplorata la funzione di condivisione file (/shared) tra la macchina fisica (host) e i dispositivi emulati.

Risposte alle Domande (Q1 - Q10)

- **Q1: Quali sono gli indirizzi IP?**
Leggendo i file .startup, l'IP di pc1 è 10.0.0.1 (con maschera /24) e l'IP di pc2 è 10.0.0.2 (con maschera /24).
- **Q2: Quali sono le tabelle di routing?**
Poiché entrambi gli host appartengono alla stessa sottorete (10.0.0.0/24) e sono connessi fisicamente allo stesso link (il dominio di collisione "A"), la tabella di routing di entrambi presenterà unicamente una rotta diretta: per raggiungere la rete 10.0.0.0/24 il traffico uscirà direttamente dall'interfaccia eth0. Non sono necessari gateway di default in questa configurazione.
- **Q3: Qual è lo stato attuale delle immagini e dei container in esecuzione prima di avviare Kathará?**
Eseguendo docker ps non dovrebbe apparire alcun container in esecuzione relativo al laboratorio. Eseguendo docker images si potrebbe vedere l'immagine kathara/base (se scaricata in precedenza o impostata come default), ma nessun componente attivo.
- **Q4: Qual è lo stato attuale dopo aver avviato Kathará?**
Dopo il comando kathara lstart, eseguendo docker ps si vedranno due container in esecuzione, corrispondenti ai dispositivi virtuali pc1 e pc2. Entrambi sono istanziati a

partire dall'immagine kathara/base.

- **Q5: Usa il comando ip per mostrare gli IP e le tabelle di routing.**

All'interno del terminale di pc1:

- IP: `ip -4 address` mostrerà 10.0.0.1/24 associato a eth0.
- Rotta: `ip route list` mostrerà una riga simile a 10.0.0.0/24 dev eth0 proto kernel scope link src 10.0.0.1.

Il risultato per pc2 sarà speculare, ma con l'IP 10.0.0.2.

- **Q6: Usando ping, verifica la connettività. Cosa succede? Perché la latenza varia nel tempo?**

Eseguendo ping 10.0.0.2 da pc1 (o viceversa), i pacchetti ICMP Echo Request riceveranno risposta (ICMP Echo Reply). Il ping va a buon fine perché i due host sono sulla stessa sottorete e collegati allo stesso dominio di collisione.

La latenza può presentare lievi fluttuazioni (jitter) perché questi non sono dispositivi hardware reali: i container condividono lo scheduling della CPU dell'OS host su cui gira Kathará, oltre ad attraversare i bridge software virtuali di Docker. Il carico momentaneo sulla macchina host fisica influisce sui tempi di risposta dei container emulati.

- **Q7: Qual è l'output di kathara linfo quando il lab è in esecuzione?**

Il comando mostrerà le informazioni del laboratorio relativo alla cartella in cui ci si trova: confermerà che il lab è "RUNNING", mostrerà i dispositivi attivi (pc1, pc2), le immagini Docker che stanno usando e a quali domini di collisione ("A") sono collegate le loro interfacce di rete.

- **Q8: Qual è l'output di kathara list quando il lab è in esecuzione?**

Fornirà un elenco a livello di sistema di tutte le macchine Kathará in esecuzione per l'utente corrente, specificando il nome della macchina (es. pc1), la cartella del laboratorio di appartenenza e lo stato del container.

- **Q9: Mostra la tabella di routing su entrambi gli host. Qual è il significato? Chi l'ha configurata?**

Come visto, la tabella mostra la rotta 10.0.0.0/24 dev eth0. Il significato è: "per parlare con qualsiasi IP compreso tra 10.0.0.1 e 10.0.0.254, butta il pacchetto direttamente sull'interfaccia eth0".

Questa rotta **non** è stata configurata esplicitamente da noi tramite un comando `ip route add`. È stata inserita *automaticamente* dal kernel Linux nel momento in cui lo script `.startup` ha assegnato un indirizzo IP con la sua subnet mask (/24) all'interfaccia di rete.

- **Q10: Qual è lo stato attuale di immagini e container dopo aver chiuso Kathará?**

Dopo aver lanciato `kathara lclean`, Kathará smonta la rete virtuale, spegne i container e li cancella. Eseguendo `docker ps` o `docker ps -a`, i container pc1 e pc2 non saranno più presenti. Tuttavia, eseguendo `docker images`, l'immagine base (kathara/base) rimarrà sul disco, pronta per essere utilizzata in laboratori futuri senza doverla riscaricare.

LAB-K3a: Basic topology and routing

Obiettivo

Apprendere e mettere in pratica i principi di base dell'indirizzamento IP e del routing statico (IPv4) costruendo topologie gradualmente più complesse: partendo da un singolo router con due host, passando per una topologia lineare (due router), fino ad arrivare ad una topologia ad anello (quattro router) analizzandone i percorsi. Inoltre, il lab introduce l'uso di strumenti di base di diagnostica di rete (ping, traceroute) e test di performance (iperf3).

Funzionamento

Il laboratorio è strutturato in tre step:

1. **Single server topology:** Collegare due host isolati tramite un router intermedio che agisce da Gateway. Vengono introdotte le reti distinte, le tabelle di routing di default per i pc e il port-forwarding indiretto.
2. **Linear topology:** Si studiano problemi di indirizzamento efficiente (VLSM) calcolando le subnets in modo da sprecare meno indirizzi IP possibili. Viene poi applicato l'instradamento in cascata (hop-by-hop) tramite percorsi statici tra due router.
3. **Routing in a loop topology:** Viene creata una maglia tra quattro router per studiare l'asimmetria del traffico. Attraverso specifiche tabelle di instradamento statico, il traffico di andata e quello di ritorno vengono forzati a compiere un percorso circolare in senso orario. Opzionalmente, viene riprodotto il problema dei "Routing Loop".

Risposte alle Domande (Part 1 - Single server topology)

- **Domande iniziali senza .startup:** Quando avvii il lab usando solo il lab.conf, **mancono gli indirizzi IP assegnati e le tabelle di routing** (sono in stato *unconfigured*). Non puoi eseguire un ping da ha a hb perché l'assenza di IP rende il livello di rete inutilizzabile (restituisce "Network is unreachable").
- **Q: Quante subnet ci sono? Come minimizzare lo spreco di IP?**
Ci sono 2 subnet (una tra ha e l'eth0 di r1, l'altra tra hb e l'eth1 di r1). In questo specifico esempio, poiché ci sono due terminali che simulano due host standard, l'utilizzo di una generica /24 (254 host) per parte (es. 192.168.1.x e 192.168.2.x) va bene. Se ha e hb fossero i soli due nodi percorsi da cavi punto a punto diretti, la sottomaschera ottima per minimizzare lo spreco sarebbe la /30 (solo 2 indirizzi IP disponibili).
- **Q1.1, Q1.2 e Q1.3: Tabelle di routing**
Configuriamo gli IP in questo modo:
 - ha: IP 192.168.1.1/24, GW 192.168.1.128
 - hb: IP 192.168.2.1/24, GW 192.168.2.128
 - r1: IP 192.168.1.128/24 (eth0), IP 192.168.2.128/24 (eth1)

Le tabelle di routing assumono quindi questo schema:**ha (Host A)**

Network prefix	Gateway	Interface
192.168.1.0/24	*	eth0
default/0	192.168.1.128	eth0

hb (Host B)

Network prefix	Gateway	Interface
192.168.2.0/24	*	eth0
default/0	192.168.2.128	eth0

r1 (Router 1)

Network prefix	Gateway	Interface
192.168.1.0/24	*	eth0
192.168.2.0/24	*	eth1

- **Q1.4: Output del Ping e cosa mancava prima**

Dopo aver inserito le rotte descritte nei file .startup, il comando ping -c 1 [IP] funzionerà in tutte le combinazioni (ha->r1, hb->r1, ha->hb). Prima dei .startup corretti, il ping "end-to-end" ha-hb non arrivava perché mancava il concetto di default gateway: l'host ha non sapeva a chi inoltrare il traffico diretto verso una rete diversa dalla propria.

- **Q1.5: Traceroute da ha ad hb e viceversa**

L'output di traceroute mostra esattamente due salti. Da ha:

1. 192.168.1.128 (router r1)
2. 192.168.2.1 (host hb)

I due percorsi (andata e ritorno) sono **identici** (simmetrici) perché, in questa particolare topologia, non esistono percorsi fisici o logici alternativi tra i due host.

- **Q1.6: Utilizzo di iPerf3**

Eseguendo iperf3 -s su hb (modalità server) e iperf3 -c 192.168.2.1 su ha (modalità client), iPerf effettuerà un test di 10 secondi e al termine riporterà la larghezza di banda TCP media (che all'interno di un'emulazione basata su container dipende fortemente dalla potenza elaborativa della macchina host che esegue Kathará, presentandosi solitamente in centinaia di Mbits/sec o alcuni Gbits/sec).

Risposte alle Domande (Part 2 - Linear topology)

Assumiamo che la matricola dia **M=10**. Dobbiamo minimizzare lo spreco di IP nel range 10.10.0.0/16. Le reti con gli host (neta1 e net2b) richiedono max 1000 host, quindi servono 10 bit dedicati agli host ($2^{10} = 1024 - 2 = 1022$ host validi). La netmask sarà $32 - 10 = 22$, ossia /22. La rete tra i due router (net12) è un collegamento punto a punto, per cui basta una maschera /30 (2 indirizzi IP validi).

- **Q2.1: Piano di indirizzamento (Addressing plan)**

Network	Network address
neta1	10.10.0.0/22
net12	10.10.8.0/30
net2b	10.10.4.0/22

Interface	IP address/netmask
ha	10.10.0.1/22
hb	10.10.4.1/22
r1a	10.10.0.2/22
r12	10.10.8.1/30
r21	10.10.8.2/30
r2b	10.10.4.2/22

- **Q2.2: Tabelle di Routing per ciascun dispositivo**

- **ha:** default/0 via 10.10.0.2 su eth0.
- **hb:** default/0 via 10.10.4.2 su eth0.
- **r1:** 10.10.4.0/22 (rete net2b) via 10.10.8.2 (interfaccia r21) su eth1.
- **r2:** 10.10.0.0/22 (rete neta1) via 10.10.8.1 (interfaccia r12) su eth0.

- **Q2.3: Traceroute**

Mostrerà i tre salti previsti: r1 -> r2 -> target.

- **Q2.4: Quale algoritmo di routing sta girando su r1 e r2? Come verificarlo?**

L'algoritmo utilizzato qui è il **Routing Statico**. Si può verificare controllando la tabella con ip route: vi si troveranno esclusivamente direttive statiche associate al protocollo base e interfacce dirette; non vi sarà traccia di protocolli di routing dinamici (come OSPF o BGP) comunemente ispezionabili o attivi da daemon FRR come vtysh.

- **Q2.5: Perché risolvere il problema di routing in questo modo (staticamente)?**

In scenari così piccoli (hub & spoke semplici, reti lineari) e completamente deterministici (esiste solo una via d'uscita e nessun percorso di fallback per la ridondanza), il routing statico è l'approccio ideale: non genera latenza dovuta al calcolo dei cammini minimi (es. Dijkstra), non causa overhead di rete scambiandosi messaggi di controllo con i vicini e aumenta la sicurezza prevenendo l'iniezione accidentale o dannosa di false route entry.

Risposte alle Domande (Part 3 - Routing in a loop topology)

Di seguito l'analisi sulla topologia ad anello fornita nella soluzione (assumendo M=10):

Range base: 10.0.0.0/8. Si richiedono max 100 host nelle LAN (basta una /25 che offre 126 ip), e /30 nei link p2p tra i quattro router nell'anello.

- **Q3.1: Piano di Indirizzamento IP**

Network	Network address	Interface	IP address/netmask
neta1	10.0.0.0/25	ha	10.0.0.1/25
net4b	10.0.0.128/25	hb	10.0.0.129/25
net12	10.0.1.0/30	r1a	10.0.0.8/25
net13	10.0.1.4/30	r12	10.0.1.1/30
net24	10.0.1.8/30	r13	10.0.1.5/30
net34	10.0.1.12/30	r21	10.0.1.2/30
		r24	10.0.1.9/30
		r31	10.0.1.6/30
		r34	10.0.1.13/30

		r42	10.0.1.10/30
		r43	10.0.1.14/30
		r4b	10.0.0.136/30

- **Q3.2: Routing per forzare la circolazione in senso orario**

Per viaggiare in senso orario, il traffico inviato da ha verso hb deve attraversare i rami r1 -> r2 -> r4. Al contrario, il traffico di ritorno inviato da hb verso ha deve attraversare l'altro lato dell'anello: r4 -> r3 -> r1.

Nel setup, r1 instrada il default verso r2 (via 10.0.1.2) e r4 instrada verso la rete base originaria 10.0.0.0/25 tramite r3 (via 10.0.1.13). Questa scomposizione crea l'asimmetria desiderata.

- **Q3.3 e Q3.4: Analisi del Ping e Traceroute**

Testando l'ambiente, il ping va a buon fine, rivelando un TTL=61 (che significa che il pacchetto è partito dal default Linux di 64 e ha attraversato esattamente 3 router, ovvero i 3 hop prestabiliti dall'anello).

Il traceroute mostra empiricamente i flussi asimmetrici.

Da ha -> hb il traffico salta sugli IP 10.0.0.8 (r1) -> 10.0.1.2 (r2) -> 10.0.1.14 (r4).

Da hb -> ha il traffico ritorna per 10.0.0.136 (r4) -> 10.0.1.13 (r3) -> 10.0.1.1 (r1).

- **Q3.5 e Q3.6 (Opzionali): L'effetto del Routing Loop**

Modificando la route su r4 e impostando erroneamente che "tutto il traffico di default" debba puntare di nuovo in alto verso r3 (tramite l'IP 10.0.1.13), non appena ha cerca di contattare un IP esterno sconosciuto (es. 1.2.3.4), il pacchetto arriva ad r1 -> r2 -> r4. Da r4 viene rispedito indietro verso r3, che lo rispedisce ad r1.

Il pacchetto entra in un circolo infinito (Routing Loop) che cessa unicamente a causa del meccanismo di protezione intrinseco nei protocolli IP: il campo *Time To Live (TTL)*. Ad ogni hop il TTL decresce di 1, giungendo a zero ed emanando un errore ICMP "Time to live exceeded". Il traceroute evidenzierà chiaramente i rimbalzi continui tra i router dello stesso anello finché l'utility non stoppa il processo.

LAB-K4: BGP

Obiettivo

Questo laboratorio introduce BGP (Border Gateway Protocol), il protocollo di routing utilizzato su Internet. Gli obiettivi principali sono: imparare a configurare sessioni di peering (eBGP) tra Autonomous System (AS) differenti, studiare il modo in cui i prefissi di rete (announcements) vengono propagati tra gli AS e analizzare la gestione delle rotte (AS-PATH) in topologie lineari e ad anello.

Funzionamento

Il lab sfrutta l'immagine kathara/frr, che integra il demone di routing FRRouting. In ogni router si definisce un file di configurazione (bgpd.conf o frr.conf) e si usa la shell avanzata vtysh per interagire con il demone BGP. Nel corso del lab:

1. Si stabiliscono sessioni TCP (porta 179) tra i router per il Peering.
2. Si "annunciano" i prefissi delle LAN connesse per diffondere la conoscenza dell'instradamento a livello globale.

Risposte alle Domande

Sezione 4.3: Peering between two routers

- **1. Come configurare i nomi degli AS e quali file vengono modificati?**
L'Autonomous System number (ASN) si configura all'interno del file della gerarchia di configurazione FRR (tipicamente /etc/frr/frr.conf o bgpd.conf a seconda del template), dichiarando ad esempio router bgp 100 su r1 e router bgp 200 su r2.
- **2. route vs ip route**
Entrambi i comandi ispezionano la tabella di routing del kernel di Linux. ip route è la versione più moderna e mostra informazioni più ricche (ad es. la sorgente della rotta, proto bgp), ma i dati di instradamento di base (prefisso e interfaccia di uscita) sono equivalenti.
- **3. Frequenza messaggi KEEPALIVE**
Ispezionando i file di log di BGP (se abilitati nel daemon), si noterà che i messaggi KEEPALIVE vengono scambiati ad intervalli regolari stabiliti nella configurazione della sessione per mantenere "established" (aperta) la connessione TCP. (Spesso il default BGP è di 60 secondi).
- **4. L'output di vtysh -> show ip bgp**
Questa schermata mostra la **RIB** (Routing Information Base) di BGP. Si evincono i prefissi di rete conosciuti, i Next Hop associati, le metriche (Weight, Local Preference) e, cosa più importante, la lunghezza e la composizione dell'AS Path per ogni prefisso.
- **5. L'output di show ip bgp summary**
Fornisce una visuale d'insieme (summary) dei peer (vicini/neighbors) attivi, mostrando gli IP remoti, la versione BGP, i contatori dei messaggi (inviati/ricevuti), il tempo da cui la sessione è attiva (Up/Down) e lo Stato/Numero di prefissi ricevuti (es. uno stato numerico "1" indica la sessione stabilita e un prefisso appreso).
- **6. Ping h1 -> h2**
In questa primissima fase, il ping **non funzionerà**. Anche se abbiamo instaurato il peering BGP tra r1 e r2, non abbiamo ancora configurato i router affinché "annuncino" (advertise) i prefissi delle proprie LAN interne (le reti 1.1.1.0/24 e 3.3.3.0/24) via BGP. r1 sa come parlare a r2, ma non sa dell'esistenza di h2.

Sezione 4.4: BGP Announcements with 2 ASs

- **1. Output di show ip route (in vtysh) vs route**
Una volta annunciati i prefissi, l'informazione è simile: vtysh mostrerà la tabella integrata di FRR (che combina le rotte connesse C e quelle apprese da BGP B), mentre route nel kernel mostrerà le stesse rotte iniettate per consentire l'effettivo data-plane forwarding.

- **2. Ping h1 -> h2**

Ora il ping va a buon fine, perché l'annuncio dei prefissi ha popolato correttamente le tabelle in tutto l'AS. r1 sa che per arrivare a 3.3.3.0/24 deve inviare il pacchetto a r2 e viceversa.

- **3. Log di BGP (Annunci inviati/ricevuti)**

Nel log si visualizzeranno messaggi di tipo UPDATE in ingresso e in uscita, che trasportano le informazioni di raggiungibilità sui prefissi 1.1.1.0/24 e 3.3.3.0/24.

- **4. AS Paths**

Osservando il lab, l'AS Path per la rete remota corrisponde al nome dell'AS in cui la rete risiede. Ad esempio su r1 (AS100):

Network prefix	AS path
1.1.1.0/24	<i>Local</i> (o vuoto/i indicante origination locale)
3.3.3.0/24	200

Sezione 4.5 e 4.6 (Topologia lineare e ad anello)

Nelle topologie ad anello o estese emerge una caratteristica essenziale di BGP riguardo ai **Routing Loops**: se nel propagarsi un annuncio ritorna ad un Autonomous System che lo aveva già processato, il router ricevente ispezionerà il campo AS-PATH; trovando il *proprio* ASN all'interno del percorso, scarterà immediatamente l'aggiornamento. In questo modo BGP **previene intrinsecamente i routing loops** su scala interdominio.

LAB-K5: BGP Multi-Homed Network

Obiettivo

Questo laboratorio introduce la configurazione di una rete di livello Enterprise o ISP di transito, in cui un AS cliente (AS 65003) è "Multi-Homed", ovvero connesso simultaneamente a due fornitori Internet (ISP1 e ISP2) per ragioni di affidabilità (backup) e ridondanza.

Funzionamento

L'obiettivo è far transitare normalmente tutto il traffico sul primario (ISP2) e riservare l'ISP1 come linea di backup in caso di fallimento della linea principale. BGP permette di fare ciò tramite il "Traffic Engineering" agendo sugli attributi:

- **Local Preference**: Usato per influenzare il traffico *in uscita* (Outbound) dall'AS 65003 verso Internet. Configurando una local-preference più alta per il neighbor ISP2 rispetto all'ISP1, i router interni del cliente sceglieranno sempre il varco primario per uscire verso Internet.
- **AS-PATH Prepending**: Usato per influenzare il traffico *in ingresso* (Inbound) da Internet

verso l'AS 65003. Poiché non possiamo imporre local preference sugli AS altrui, possiamo artigianalmente "allungare" l'AS Path annunciato sull'interfaccia verso l'ISP1 (ripetendo più volte l'AS 65003 65003 65003). In questo modo la porzione Internet vedrà la rotta per arrivare all'ISP2 come più breve, usandola come path primario.

Esperimenti con Fallimenti di Rete (Sezione 5.3)

Spegnendo la sessione o i link emerge la rapidità (e i limiti temporali) della convergenza di BGP:

1. **Chiusura Neighbor via software:** Eseguendo `neighbor <IP> shutdown` in vtysh, BGP scambia messaggi formali per chiudere la sessione (invia un NOTIFICATION) e invia subito messaggi di withdraw ritirando i prefissi. La rete transita velocemente sul link di backup (ISP1) quasi istantaneamente dopo l'elaborazione del withdraw.
2. **Spegnimento link fisico (ip link set ethX down):** A differenza della disconnessione soft (graceful), il down improvviso può dipendere dai timer. Se il router capisce subito che il link layer è andato (Carrier down), la convergenza è veloce. Se si perdono pacchetti (es. hub guasto) e BGP deve attendere il timeout dei messaggi Keepalive (Spesso 180s o 90s di Hold Time), il ripristino della connettività sul backup avverrà con notevole ritardo. Alla fine la rotta scadrà, e BGP dirotterà il traffico sull'ISP1 assicurando la ridondanza.

LAB-K6: BGP in a data center

Obiettivo

I Data Center moderni di grandissime dimensioni usano BGP anche internamente, sostituendo protocolli come OSPF grazie alla robustezza superiore. L'obiettivo è realizzare una topologia "Clos" o Multilayer (Leaf-Spine-ToF) sfruttando intensamente eBGP ed abilitando il bilanciamento del carico a costo uguale (ECMP).

Funzionamento

La rete è gerarchica: i server sono collegati a router *Leaf*; questi ai router *Spine*, i quali scalano superiormente verso i *Top of Fabric (ToF)*. Si utilizzano indirizzi /30 punto a punto tra ciascuno strato.

Tramite i file `bgpd.conf` analizzati (Q1-Q8), emergono queste considerazioni sulla configurazione BGP:

- Invece di ripetere blocchi per ogni neighbor, vengono estensivamente usati i **peer-group**: si definisce un gruppo, si applicano le policy BGP a quel gruppo e poi si dichiarano i vari neighbor appartenenti al gruppo. Questo pulisce il codice di configurazione nei ToF/Spine dove vi sono svariate interfacce.
- Ogni interruttore/router ha un ASN privato distinto e un proprio Router-ID univoco (spesso derivato dall'interfaccia Loopback).

Multilayer in esecuzione (Running - Q9-14)

- I ping (es. da Leaf 1 a Leaf 2 attraverso Spine o ToF) vanno a buon fine: i percorsi completi

sono annunciati.

- Eseguendo `show ip bgp`, si noterà che per un medesimo prefisso remoto (es. la subnet di un Leaf), la RIB di un ToF conterrà **percorsi multipli validi**. Essendo l'AS-PATH lungo in egual misura (stesso hop-count gerarchico della struttura Clos), entrano in gioco le dinamiche di pareggio.
- **ECMP (Equal-Cost Multi-Path)**: Abilitandolo, BGP non seleziona più *una* sola best path scartando le altre, ma inietta nella tabella di routing del kernel *tutti* i percorsi validi.
- Il bilanciamento ECMP (invio traffico dai server) può avvenire basandosi su un hash a 5 tuple (IP Sorgente, IP Dest, Porta Sorgente, Porta Dest, Protocollo). Inviare più flussi TCP differenti farà sì che i pacchetti attraversino link Spine/ToF diversi, usando l'intera capacità in parallelo.
- **Gestione Guasti (Failures)**: Abbattendo un'interfaccia fisica con il comando `ip link set ethX down`, BGP disattiva il path guasto. Se il traffico stava sfruttando l'ECMP tramite più Spine, il path difettoso viene rimosso in modo fulmineo dal kernel; la connettività tra i server *non cade*, riducendosi unicamente la capacità di banda passante totale (degradazione morbida) a dimostrazione della validità architetturale della topologia Clos.

LAB-K7: Two levels load balancing for a web service

Obiettivo

Questo lab esamina l'architettura classica ad alta affidabilità dietro a servizi web famosi, sfruttando due diversi strati di "bilanciamento di carico" in simultanea:

1. **Livello 1: DNS Load Balancing** (operante al layer applicativo mediante record mappati multipli).
2. **Livello 2: Layer-4 Load Balancing** (operante ai livelli di Trasporto TCP/UDP instradando fisicamente la connessione).

Funzionamento e Risposte

5.1 Setup ed Elementi Base

- **1. Quale file contiene la configurazione per il load balancing DNS? Perché funziona?**
Il file di zona (es. all'interno del server DNS autoritativo, in `/etc/bind/db.uniroma3.it`) contiene più record di tipo A associati allo stesso nome a dominio (ad es., due IP pubblici diversi per `www.uniroma3.it` riferiti a lb1 e lb2). Il DNS opera tramite la tecnica del **Round-Robin**: restituisce l'intera lista di IP ma ne mischia/ruota l'ordine ad ogni query. Il client sceglierà solitamente il primo della lista per la connessione.
- **2. Quale file contiene il bilanciamento di Layer-4? Come funziona?**
Sui nodi di load balancing (es. lb1 e lb2), vi è un software specifico (come HAProxy o Linux Virtual Server) le cui configurazioni o regole iptables dirottano trasparentemente i pacchetti TCP in arrivo (porta 80) verso i web server reali sul back-end interno (ws1, ws2, ws3). I LB L4 ruotano le connessioni TCP distribuendo pariteticamente i client.

5.2 e 5.3 Test ed Analisi

- Lanciando richieste col browser da riga di comando links `www.uniroma3.it` e forzando l'aggiornamento, la connessione verrà instaurata verso un server backend diverso di volta in volta (confermabile estraendo le statistiche sui file serviti o dal terminale). La combinazione L4 Round-Robin bilancia uniformemente ad ogni *TCP handshake*.
- Esaminando lo sniff del traffico in Wireshark durante un'operazione client, si notano in chiaro le query DNS ricorsive di `ldns` verso il server root/autoritativo che rispondono elencando i vari IP dei Load Balancer, a conferma dell'operatività del livello DNS.

5.4 Esperimenti su Guasti Macchine (Failures)

1. **Guasto a un Web Server (es. ws3):** Se abbattiamo ws3, il client che vi incappa potrebbe ricevere un errore temporaneo. Tuttavia, se i load balancer layer-4 sono dotati di "*Health Checks*" (controlli attivi sullo stato HTTP del backend), capiranno velocemente che ws3 non risponde (503 / timeout) e lo rimuoveranno dal pool del Round-Robin, indirizzando il traffico solo su ws1 e ws2. L'utente finale non percepirà quasi il problema.
2. **Guasto a un Load Balancer (es. lb1):** Se lb1 muore e si rimuove dal DNS (`dnsu3`) il record di pertinenza, c'è un problema latente. Tutti i *nuovi* client interrogheranno il DNS, otterranno solo il record superstite lb2 e navigheranno tranquillamente. Viceversa, per i client che avevano **già risolto** l'IP nei minuti antecedenti al guasto, l'indirizzo obsoleto morto (lb1) permarrà nella loro cache DNS locale per tutta la durata del **TTL (Time to Live)**. Finché la cache non scadrà, il browser fallirà nel connettersi restituendo pagina irraggiungibile, evidenziando il principale svantaggio del DNS Load balancing in caso di disastri.

LAB-KAAS: Katharà as a Service

Obiettivo

Questo progetto (KaaS) ha come obiettivo quello di implementare un'interfaccia containerizzata (Web-based) per Katharà, consentendo l'esecuzione e la gestione dei laboratori in modo molto più accessibile tramite browser. Mira a fornire una vera e propria interfaccia di progettazione in cui mappare visivamente (drag-and-drop) nodi e connessioni direttamente nella notazione `lab.conf`.

Funzionamento e Struttura

L'architettura alla base di KaaS è organizzata in svariati servizi coordinati:

1. **Traefik (Load Balancer):** Usato come reverse proxy interno per gestire le comunicazioni e risolvere i nomi locali verso domini fittizi (es. `kaas.test`, `tty.kaas.test`).
2. **Docker-in-Docker (dind):** Il nucleo del sistema. Esegue i laboratori Katharà non direttamente sul sistema host, ma all'interno di una bolla Docker protetta e isolata.
3. **Web-Interface e REST API:** Frontend che genera visivamente la mappa del laboratorio e la comunica via API (`rest.kaas.test`) al client Docker sottostante.
4. **Interfaccia TTY (Web Terminal):** Fornisce all'utente l'accesso terminale (shell) interattivo diretto ai nodi virtuali del laboratorio direttamente dalla finestra del browser, sfruttando

socket.io.

Si avvia in ambiente di test locale con il semplice stack composito (es. `kaas up -d` tramite script shell personalizzati) che eroga tutto l'ecosistema necessario sul proprio computer.

LAB: QUIC and Congestion Control

Obiettivo

Sperimentare e comprendere nella pratica il funzionamento degli algoritmi di *Congestion Control* (controllo della congestione) che stanno alla base di Internet moderno. Nel laboratorio si mettono a confronto i protocolli di trasporto classici (TCP, UDP) e le moderne implementazioni basate su QUIC, al fine di analizzare la reazione alle perdite di pacchetti, la fairness nella condivisione dei link e l'impatto del Round Trip Time (RTT).

Funzionamento

Viene proposta una topologia a collo di bottiglia (*bottleneck*): due client tentano di fare l'upload simultaneo di file verso un server, passando per un link strozzato con banda limitata a ~8Mbps. Per misurare e analizzare i risultati:

- Si utilizza la formattazione di logging **QLOG** (standard in via di sviluppo IETF), specifica per le metriche interne di QUIC. Questi file vengono poi passati in pasto a script Python (librerie *pandas* e *numpy*) in grado di parsare i dati e generare grafici (*plot*) sull'evoluzione delle variabili di congestione nel tempo.
- **Sniffing con Wireshark:** Essendo QUIC un protocollo pesantemente criptato basato su TLS (non cripta solo i payload, ma anche grossa parte degli header), Wireshark mostrerà il traffico come indecifrabile. Per aggirare l'ostacolo in ambiente di laboratorio, all'avvio del client si esportano in chiaro le chiavi segrete del TLS (parametro `--keylog-file`). Queste chiavi, salvate in `shared/keys.txt`, vengono poi importate manualmente in Wireshark (nelle preferenze del protocollo TLS -> *(Pre)-Master-Secret log filename*), rivelando i payload e rendendo possibile l'analisi minuziosa dei flussi.

LAB: OSPF I (Single Area)

Obiettivo

Costruire le fondamenta del protocollo di routing link-state OSPF configurandolo su una topologia a singola area di backbone (Area 0.0.0.0). Si studierà l'uso del database di routing (LSDB), i criteri di elezione dei Designated Router (DR/BDR) sui link broadcast e la risposta della rete al variare dei costi logici delle interfacce.

Funzionamento e Considerazioni Principali

Sui container viene usato FRRouting (FRR) modificando `frr.conf` in `/etc/frr/` e dicendo al protocollo su quali reti girare (es. `network 10.0.0.0/16 area 0.0.0.0`).

Task e Domande chiave:

- **Q1-Q4 (Comandi OSPF):** * show ip ospf neighbor fa vedere l'elenco dei router direttamente adiacenti e il loro stato (es. *Full/DROther, Full/DR*).
 - show ip ospf database stampa il Link State Database (LSDB) contenente gli LSA ricevuti, essenziale per costruire l'albero di Dijkstra.
 - show ip ospf route mostra l'instradamento esclusivo dal punto di vista dell'algoritmo OSPF (alcune rotte potrebbero non essere nella tabella globale dell'OS se soppiantate da rotte statiche migliori).
- **Q8 (Modifica Costi):** Modificando il comando ospf cost X su una interfaccia, i router ricalcolano (tramite l'algoritmo SPF) il percorso minimo. Si osserva un aggiornamento degli LSA che riflette la nuova mappa. Il tempo di convergenza dipende dai timer configurati (es. LSA Flood e ricalcolo SPF), ma di base è estremamente veloce su una rete a singola area, assestandosi spesso in meno di pochi secondi se c'è link up/down diretto.
- **Q9 (Elezione DR):** Su ogni segmento di rete Broadcast (come uno switch condiviso) viene eletto un DR. Le modifiche sono tenute "infrequenti" intenzionalmente per stabilità: una volta che un DR è stabile, i nuovi router inseriti si adeguano per non forzare costantemente ricalcoli distruttivi in caso di instabilità.
- **Q11 (Perdita di Pacchetti):** Aggiungendo un packet loss artificiale al router tramite kernel Linux (tc qdisc netem loss 1%), si perturba la rete. All'1% le ritrasmissioni OSPF riescono ancora a compensare, ma alzando i valori, i pacchetti di Hello o Dead Interval andranno irrimediabilmente persi; le adiacenze cadranno e riappariranno di continuo ("flapping"), generando repentine variazioni nell'LSDB e un ricalcolo faticoso o instabile (convergenza lenta o assente).

LAB: OSPF II (Multi-Area)

Obiettivo

Superare i limiti di scalabilità dell'OSPF a singola area implementando gerarchie Multi-Area: la Backbone Area (0.0.0.0) e due aree separate (1.1.1.1 e 2.2.2.2) concepite come Stub Areas (aree in cui confluiscono solo rami morti). Vengono analizzati nel dettaglio i ruoli dei router di frontiera ABR (Area Border Router) e ASBR (Autonomous System Boundary Router).

Funzionamento e Considerazioni Principali

La topologia richiede di confinare gli LSA più granulari solo all'interno delle loro aree locali, propagando esternamente solo le rotture riassuntive tramite il backbone.

Task e Domande chiave:

- **Q1 (Database Multi-area):** Eseguendo show ip ospf database summary, emergono i *Summary LSA (Type 3)*, che gli ABR generano per dire: "Io so come raggiungere questa subnet nell'altra area; per andarci, passate da me".
- **Q2 (Route Inter-Area):** Il comando show ip ospf route etichetterà questi percorsi esterni alla propria area come **IA (Inter-Area)**, differenziandoli palesemente dalle rotte locali.

- **Q3 (Adiacenze):** Un router interno a un'area (es. 1.1.1.1) formerà adiacenze *solo* con i router collegati al suo stesso segmento (come l'ABR bb1 sull'interfaccia speculare). Non vi sarà mai un'adiacenza OSPF tra router di aree diverse se non sono fisicamente connessi su una stessa sottorete condivisa (il compito spetta infatti all'ABR unire i pezzi).
- **Q5 (LSDB ABR):** Un ABR (essendo il ponte tra due mondi) partecipa in due o più aree. Osservando il database con `show ip ospf database router`, si vedrà chiaramente che l'ABR deve mantenere *due Link State Database separati*: uno completo per l'Area 0 e uno per l'Area 1, unificandoli solo tramite LSA riassuntivi.
- **Q8 (Stub Default Route):** In un'area "Stub", i router interni (r2) sono impossibilitati per statuto a ricevere gli LSA esterni (Type 5). Invece di avere in pancia le migliaia di indirizzi dell'AS esterno, l'ABR vi inietta automaticamente una comoda Default Route (0.0.0.0/0) (spesso Type 3). Il router la sfrutterà con la sua normale metrica OSPF uscendo ciecamente verso l'ABR.
- **Q11 & Q12 (Total Stub e ASBR esterno via BGP):** * Trasformando l'area in stub no-summary (Totalmente Stub), vengono bloccati persino gli LSA Inter-area (Type 3): entra unicamente la default route. Ottimo per risparmiare risorse.
 - Inserendo un router che impara via BGP rotte esterne e le inietta in OSPF tramite ridistribuzione (ASBR), tali rotte appariranno come **E2** (External Type 2, la cui particolarità è propagare il costo con valore fisso senza sommare il costo interno al link OSPF). Le aree stub bloccheranno questi aggiornamenti LSA Type 5; l'unico modo in cui il traffico stub potrà uscire verso la rotta BGP è, ancora una volta, sfruttando l'onnipresente default route OSPF dell'ABR.